

PDF Chatbot

Design Document

A full-stack Retrieval-Augmented Generation (RAG) application for intelligent PDF document querying, built with Next.js, LangGraph, and Supabase.

Property	Value
Stack	Next.js 14 + LangGraph + Supabase + OpenAI
Architecture	Monorepo (Turborepo) with separate frontend/backend
Language	TypeScript (full stack)
LLM	GPT-4o-mini (configurable)
Vector DB	Supabase pgvector
Date	April 2026

Contents

- 1. System Overview
- 2. Architecture Diagram
- 3. Frontend Architecture
- 4. Backend Architecture
- 5. PDF Ingestion Pipeline
- 6. RAG Query Pipeline
- 7. Data Layer
- 8. Key Dependencies
- 9. Configuration & Environment

10. Testing Strategy

11. Deployment

1. System Overview

The PDF Chatbot is a Retrieval-Augmented Generation (RAG) application that allows users to upload PDF documents and ask natural-language questions about their content. The system intelligently routes queries, retrieves relevant document chunks via vector similarity search, and generates concise answers with source citations.

Key Capabilities

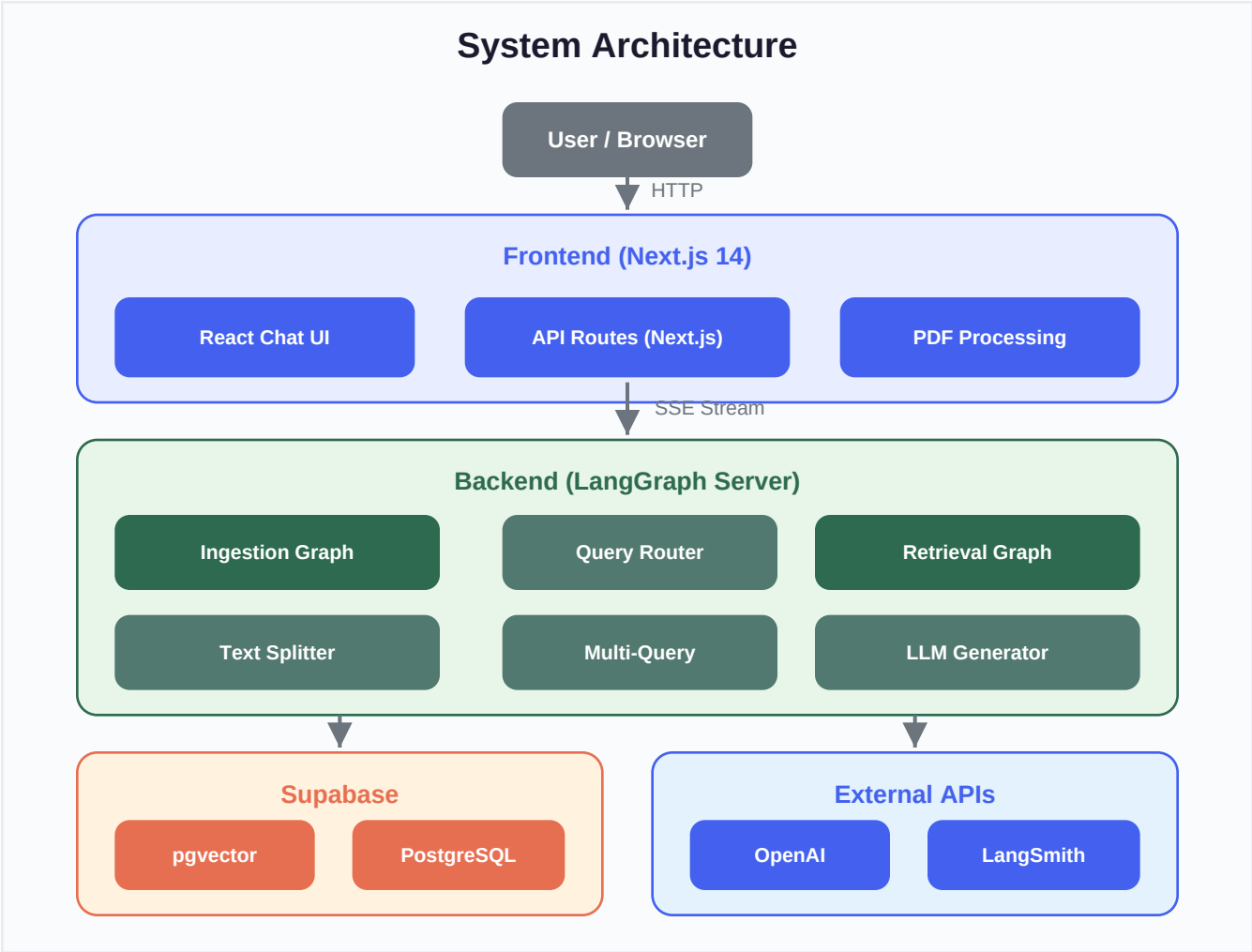
- **PDF Ingestion** - Upload up to 5 PDFs (10MB each), automatically chunked and embedded
- **Intelligent Routing** - LLM-based query classification (retrieve vs. direct answer)
- **Multi-Query Retrieval** - Generates 3 query variants for better recall
- **Streaming Responses** - Real-time token streaming via Server-Sent Events (SSE)
- **Source Citations** - Expandable accordion showing retrieved document chunks with page numbers
- **Persistent Threads** - Conversation history preserved per LangGraph thread

Project Structure

```
pdf-chatbot/ # Turborepo monorepo root
backend/ # LangGraph server (Node.js 20)
src/ingestion_graph/ # PDF ingestion state machine
src/retrieval_graph/ # RAG query state machine
src/shared/ # Shared retriever, state, config
frontend/ # Next.js 14 app
app/api/ingest/route.ts # PDF upload endpoint
app/api/chat/route.ts # Chat streaming endpoint
components/ # React UI components
lib/ # LangGraph client wrapper
constants/ # Graph configuration
```

2. Architecture Diagram

The system follows a three-tier architecture: a React-based frontend communicates with LangGraph state machines via Next.js API routes, which in turn interact with Supabase (vector storage) and OpenAI (embeddings & LLM).



Component Responsibilities

Component	Responsibility
React Chat UI	Client-side chat interface with file upload, message rendering, streaming display
API Routes	Next.js server routes bridging frontend to LangGraph; handles PDF parsing
PDF Processing	Uses PDFLoader to extract text + metadata from uploaded files
Ingestion Graph	LangGraph state machine: splits text into chunks, generates embeddings, stores in Supabase

Component	Responsibility
Retrieval Graph	LangGraph state machine: routes queries, retrieves docs, generates answers
Query Router	LLM-based classifier deciding retrieve vs. direct answer
Multi-Query	Generates 3 alternative query formulations for improved retrieval recall
Supabase pgvector	PostgreSQL with vector extension for similarity search
OpenAI	Provides text-embedding-3-small for embeddings and GPT-4o-mini for generation
LangSmith	Optional tracing and observability for debugging LangGraph runs

3. Frontend Architecture

The frontend is a Next.js 14 application using the App Router pattern. It provides a single-page chat interface with PDF upload, real-time streaming responses, and source citation display.

3.1 Technology Stack

Technology	Details
Framework	Next.js 14.2 (App Router)
Language	TypeScript 5
UI Components	Radix UI / shadcn/ui (60+ components)
Styling	Tailwind CSS 3
Icons	Lucide React
Forms	React Hook Form + Zod validation
Notifications	Sonner (toast)
Theme	next-themes (dark mode support)

3.2 Key Files

File	Description
app/page.tsx	Main chat interface (client component). Manages thread creation, message state, file upload, streaming, and source display.
app/api/ingest/route.ts	POST endpoint for PDF upload. Parses PDFs via PDFLoader, invokes ingestion_graph on LangGraph server.
app/api/chat/route.ts	POST endpoint for chat queries. Streams LangGraph retrieval_graph output as SSE to the client.
components/ChatMessage.tsx	Renders individual messages with copy button and expandable source citations accordion.
components/FilePreview.tsx	Displays uploaded file cards with name, size, and remove button.
components/ExamplePrompts.tsx	Shows suggested prompts for new users to get started quickly.
lib/langgraph-base.ts	LangGraph SDK client wrapper: createThread, getThread, getThreadState, etc.
constants/graphConfigs.ts	Configuration objects for retrieval and ingestion graphs (model, k, provider).

3.3 State Management

The frontend uses React local state (useState/useRef) rather than a centralized store. Key state includes:

- **messages[]** - Chat history (user + assistant messages)
- **files[]** - Currently selected PDF files for upload
- **threadId** - Active LangGraph thread ID (created on mount)
- **isStreaming** - Whether a response is currently streaming
- **lastRetrievedDocsRef** - Ref holding retrieved source documents for the latest response
- **AbortController** - Enables cancellation of in-flight streaming requests

3.4 Streaming Protocol

The chat endpoint returns Server-Sent Events (SSE). The frontend reads the stream via ReadableStream and processes two event types:

- **messages/partial** - Incremental token updates for real-time typing effect
- **updates** - State updates containing retrieved documents for source citations

4. Backend Architecture

The backend runs on a LangGraph server (Node.js 20) and exposes two state machine graphs: one for document ingestion and one for retrieval-augmented generation. Both graphs are defined in TypeScript using LangGraph's StateGraph API.

4.1 LangGraph Configuration

```
langgraph.json:
node_version: "20"
graphs:
  ingestion_graph: "./src/ingestion_graph/graph.ts:graph"
  retrieval_graph: "./src/retrieval_graph/graph.ts:graph"
```

4.2 Shared Components

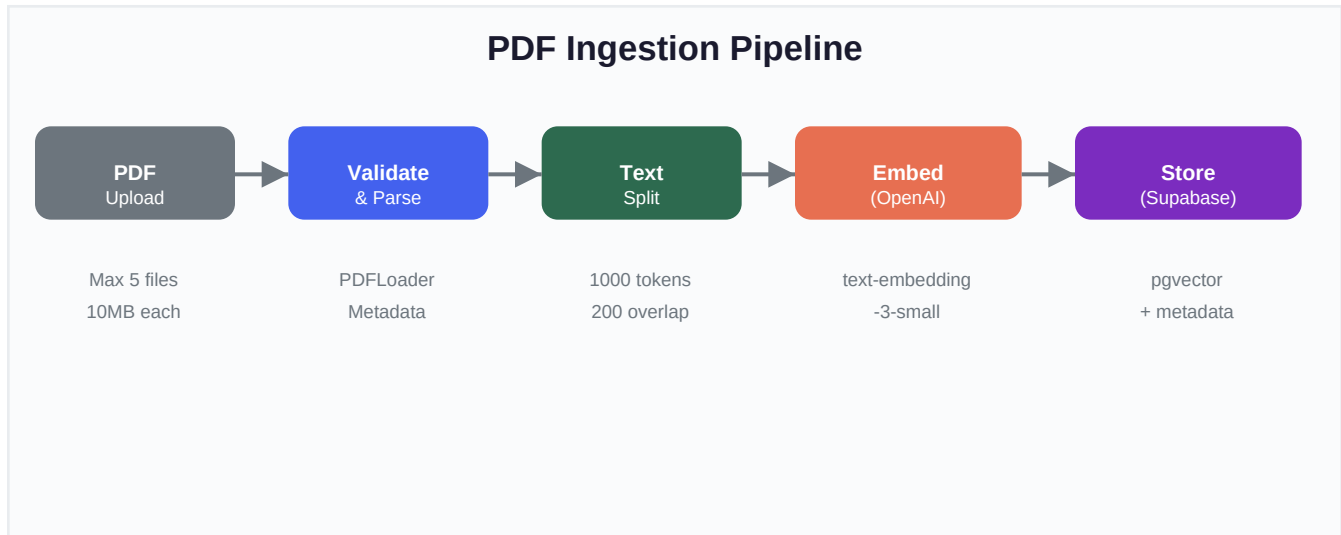
File	Purpose
shared/retrieval.ts	makeRetriever() factory - creates SupabaseVectorStore with OpenAI embeddings
shared/state.ts	Shared state annotations (IndexStateAnnotation, AgentStateAnnotation)
shared/configuration.ts	Base configuration: retrieverProvider, k, filterKwargs

4.3 Prompt Templates

Prompt	Role
ROUTER_SYSTEM_PROMPT	Routing assistant that classifies queries as 'retrieve' or 'direct'
MULTI_QUERY_PROMPT	Generates 3 alternative formulations of the user's question
RESPONSE_SYSTEM_PROMPT	QA assistant - answers in max 3 sentences using provided context

5. PDF Ingestion Pipeline

The ingestion pipeline converts uploaded PDF files into searchable vector embeddings stored in Supabase. It is implemented as a LangGraph state machine with a single node.



5.1 Ingestion Graph State

```
IndexStateAnnotation {  
  docs: Document[] // Input: parsed PDF documents  
}
```

Graph Flow: START -> ingestDocs -> END

5.2 Step-by-Step Process

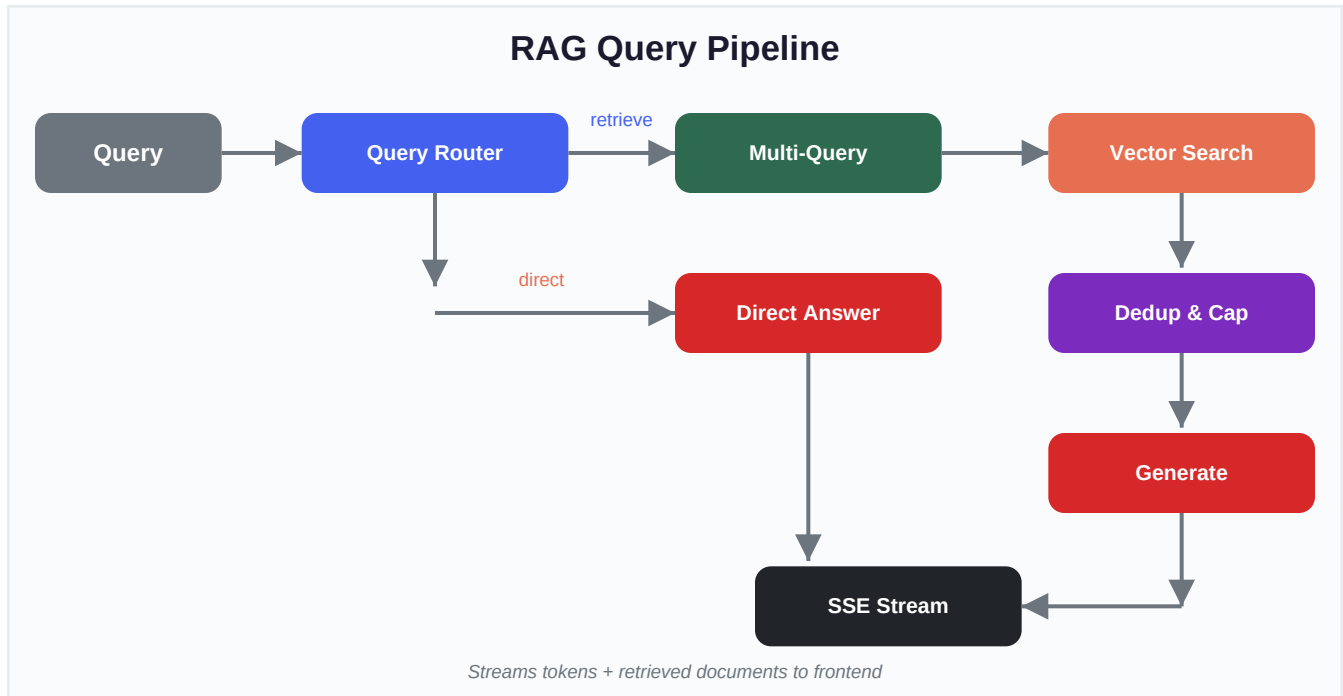
- **Step 1 - File Upload (Frontend):** User selects PDFs via file input. Validated: PDF type, max 10MB, max 5 files. Sent as FormData to /api/ingest.
- **Step 2 - PDF Parsing (API Route):** Each file is converted to a Buffer, written to a temp file, and processed by LangChain PDFLoader. Extracts text with metadata (filename, page number). Temp files are cleaned up.
- **Step 3 - Graph Invocation:** All parsed documents are sent to the ingestion_graph via LangGraph SDK runs.wait() method.
- **Step 4 - Text Splitting:** RecursiveCharacterTextSplitter divides documents into chunks of 1000 tokens with 200-token overlap for context continuity.
- **Step 5 - Embedding Generation:** Each chunk is embedded using OpenAI text-embedding-3-small model (1536 dimensions).
- **Step 6 - Vector Storage:** Embeddings are stored in Supabase documents table with pgvector extension. Metadata (filename, page, line range) is preserved as JSONB.

5.3 Configuration

Parameter	Default	Description
retrieverProvider	'supabase'	Vector store backend
useSampleDocs	false	Load bundled sample docs instead of user uploads
docsFile	path	Location of sample docs JSON file

6. RAG Query Pipeline

The retrieval graph implements an intelligent query pipeline with routing, multi-query expansion, parallel retrieval, deduplication, and streaming response generation.



6.1 Retrieval Graph State

```
AgentStateAnnotation {  
  query: string // User's question  
  route: 'retrieve' | 'direct' // Routing decision  
  messages: AIMessage[] // Conversation history  
  documents: Document[] // Retrieved context  
}
```

6.2 Node Descriptions

checkQueryType (Router)

Uses the LLM with structured output (Zod schema) to classify the query. If the question relates to uploaded documents, it routes to 'retrieve'. For general knowledge questions, it routes to 'direct'.

retrieveDocuments

Implements multi-query retrieval for improved recall:

- Generates 3 alternative formulations of the user's question using MULTI_QUERY_PROMPT

- Performs parallel vector similarity searches for all query variants
- Deduplicates results while preserving relevance ordering
- Caps results at k*2 documents (default: 10)

generateResponse

Formats retrieved documents as XML context blocks, combines with conversation history, and calls the LLM. The RESPONSE_SYSTEM_PROMPT instructs the model to answer in max 3 sentences. Response tokens are streamed back via SSE.

answerQueryDirectly

For queries that don't need document retrieval (e.g., 'hello', 'what can you do?'), the LLM answers directly without context. This avoids unnecessary vector searches.

6.3 Configuration

Parameter	Default	Description
queryModel	'openai/gpt-4o-mini'	LLM for routing and generation
retrieverProvider	'supabase'	Vector store backend
k	5	Number of documents per query variant
filterKwargs	{}	Optional metadata filters for retrieval

7. Data Layer

7.1 Supabase (Vector Store)

The application uses Supabase PostgreSQL with the pgvector extension for vector similarity search. Documents are stored with their embeddings and metadata.

Table Schema

Column	Type	Description
content	TEXT	Document chunk text
embedding	vector(1536)	OpenAI embedding (text-embedding-3-small)
metadata	JSONB	Filename, page number, line range, etc.
created_at	TIMESTAMP	Auto-generated insertion time

Database Function

```
match_documents(query_embedding, match_count, filter)
```

A Supabase RPC function that performs vector similarity search. Returns the top match_count documents closest to the query embedding, with optional metadata filtering.

7.2 Embedding Model

Property	Value
Model	text-embedding-3-small
Dimensions	1536
Provider	OpenAI
Usage	Document chunking (ingestion) + query embedding (retrieval)

7.3 LangGraph Thread Persistence

LangGraph manages thread state (conversation history, retrieved documents) server-side. Each browser session creates a new thread on mount. The thread ID is used to maintain multi-turn conversation context.

8. Key Dependencies

8.1 Frontend Dependencies

Package	Version	Purpose
next	14.2.16	React framework (App Router)
react	18	UI library
tailwindcss	3	Utility-first CSS
@radix-ui/*	various	Headless UI components (shadcn/ui)
@langchain/langgraph-sdk	0.0.42	LangGraph server client
@langchain/core	0.3.40	LangChain core abstractions
@langchain/community	0.3.31	Community integrations (PDFLoader)
pdf-parse	1.1.1	PDF text extraction
react-hook-form	7.54.1	Form state management
zod	3.24.1	Schema validation
sonner	1.7.1	Toast notifications
next-themes	0.4.4	Dark mode support

8.2 Backend Dependencies

Package	Version	Purpose
@langchain/langgraph	1.2.8	Graph orchestration framework
@langchain/openai	1.4.3	OpenAI LLM + embeddings integration
@langchain/core	1.1.39	Core LangChain abstractions
@langchain/community	1.1.27	Community integrations
@supabase/supabase-js	2.48.1	Supabase client (vector store)
pdf-parse	1.1.1	PDF text extraction
zod	3	Schema validation (structured output)
uuid	11.0.5	Unique ID generation
chromadb	1.10.4	Alternative vector DB (optional)

9. Configuration & Environment

9.1 Environment Variables

Variable	Scope	Description
OPENAI_API_KEY	Backend	OpenAI API key for embeddings and LLM
SUPABASE_URL	Backend	Supabase project URL
SUPABASE_SERVICE_ROLE_KEY	Backend	Supabase service role key (server-side only)
NEXT_PUBLIC_LANGGRAPH_API_URL	Frontend	LangGraph server URL (default: http://localhost:2024)
LANGCHAIN_API_KEY	Both	LangChain/LangSmith API key
LANGCHAIN_TRACING_V2	Both	Enable LangSmith tracing (true/false)
LANGCHAIN_PROJECT	Both	LangSmith project name
LANGGRAPH_INGESTION_ASSISTANT_ID	Frontend	Graph name for ingestion (ingestion_graph)
LANGGRAPH_RETRIEVAL_ASSISTANT_ID	Frontend	Graph name for retrieval (retrieval_graph)

9.2 Graph Runtime Configuration

Passed via the *configurable* field in LangGraph run requests:

Config Object	Values
retrievalAssistantStreamConfig	queryModel: 'openai/gpt-4o-mini', retrieverProvider: 'supabase', k: 5
indexConfig	useSampleDocs: false, retrieverProvider: 'supabase'

10. Testing Strategy

Tests are located in backend/ __tests__ / and run with Jest. The CI pipeline runs a subset of tests (state + prompt) on every push and PR.

Test File	Type	Coverage
ingestion_graph/state.test.ts	Unit	Document reducer logic and state transitions
retrieval_graph/integration.test.ts	Integration	End-to-end RAG flow: routing, retrieval, generation
retrieval_graph/promptTemplate.test.ts	Unit	Prompt template formatting and variable substitution

10.1 CI/CD Pipeline

GitHub Actions workflow (.github/workflows/ci.yml):

- Triggered on push to main and pull requests
- Uses Node.js 22.x
- Runs backend linting (yarn lint)
- Runs state and prompt template tests
- Integration tests require OPENAI_API_KEY + SUPABASE credentials (run locally)

11. Deployment

11.1 Local Development

```
# Start LangGraph backend server
cd backend && yarn langgraph:dev # http://localhost:2024

# Start Next.js frontend
cd frontend && yarn dev # http://localhost:3000
```

11.2 Architecture Notes

- **Backend** - LangGraph server runs as a standalone process on port 2024. Configured via langgraph.json. Docker-ready (.dockerignore present).
- **Frontend** - Standard Next.js deployment. API routes act as a bridge to the LangGraph server, handling PDF parsing and SSE streaming.
- **Supabase** - Hosted PostgreSQL with pgvector. Requires the documents table and match_documents function to be provisioned.

- **Observability** - LangSmith tracing enabled via `LANGCHAIN_TRACING_V2`. Traces all LangGraph runs for debugging and monitoring.